End-to-End Text-to-Speech Synthesis

A replication of SVR, neural-network and seq2seq-with-attention models

Akula Pragnya (PES1UG24AM028) · Akshata Amara (PES1UG24AM025)

Replicating Wang, Yang & Li, "End to End Text to Speech Synthesis" (CS 229, Stanford, 2018)

1. Problem Statement

- TTS turns written text into audible speech; classical systems chain hand-built stages,
- Assigned problem: replicate Wang, Yang & Li, “End to End Text to Speech Synthesis”
- Build a pipeline: sentence -> predicted magnitude spectrogram (learned model) -> waveform.
- Compare three models of increasing capacity:
  - Support Vector Regression (SVR)
  - A simple neural network
  - A seq2seq model with attention

2. Dataset

LJ Speech corpus

13,100 clips, single speaker reading non-fiction passages

Public-domain (Ito & Johnson, 2017)

200-sentence subset used, as in the original paper

Clip selection

<= 50 characters and <= 3.5 s after silence trimming

Fixed random shuffle, seed 0

Split: 190 training sentences / 10 held-out validation

Text processing

Each word -> CMU-dict phonemes OR letters (p = 0.5, random per word)

Punctuation kept; symbols -> integer IDs

Zero-padded to 96 symbols => input shape 200 x 96

Audio processing

22,050 Hz; FIR pre-emphasis y[t] = x[t] - 0.97 x[t-1]

STFT: window 2,048, hop 512 -> 1,025 frequency bins/frame

scaled to [0,1], zero-padded to 152 frames

=> target shape 200 x 152 x 1,025

3. Approach - The Pipeline

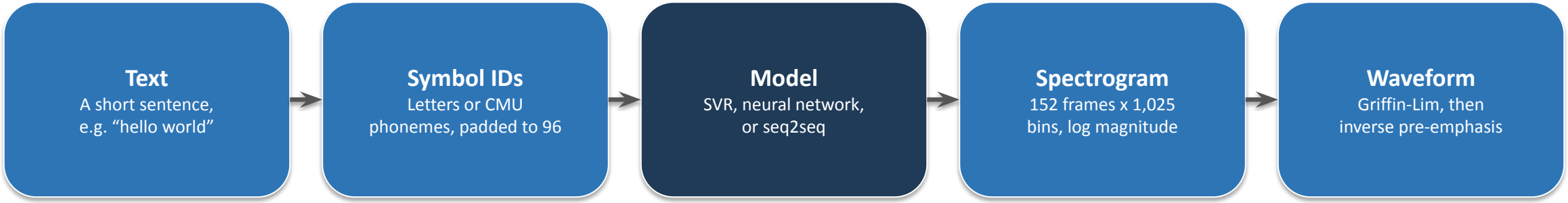


Figure 1. Only the Model stage is learned; the other stages are fixed signal and text processing.

- Three stages, all models trained on the same data and the same split:

– text -> symbol IDs
– symbol IDs -> 152 x 1,025 spectrogram (the learned part)
– spectrogram -> waveform

3. Models — Baselines

SVR (baseline 1)

Support vector regression, polynomial kernel
degree = 3, C = 1, epsilon = 0.01, standardised inputs
Paper trains one multi-output SVR per time step
(a 1,025-output SVR per frame was too slow)

So: compress every frame to 32 PCA components, fit one SVR per (frame, component) => 152 x 32 = 4,864 models
Training time: 22 s on a laptop CPU

Neural network (baseline 2)

32-dimensional embedding, flattened
Fully connected sigmoid layer, 256 units
Linear layer -> 152 x 1,025 values (“stretched” spectrogram)
Sigmoid output
MSE loss, Adam (lr = 1e-3), mini-batches of 10, 200 epochs
Training time: 608 s (~10 min) on a laptop CPU

Deviation from the paper: PCA compression (32 components) in place of a direct 1,025-output regressor per frame.

3 · Models — Seq2seq & Waveform Reconstruction

Seq2seq with attention (advanced)

128-dimensional embedding; LSTM encoder, 512 units

LSTM decoder, 512 units, with additive attention (Bahdanau et al.) over the encoder states

Produces one frame per step from the previous frame, via a 256-unit pre-net

Trained with teacher forcing (true previous frame as input);

MSE loss, Adam, batch size 10, gradient clipping at 1.0, 150 epochs

At inference: feeds back its own predictions

Training time: 4,640 s (~77 min) on a laptop CPU

Waveform reconstruction

Only magnitudes are predicted -> phase estimated with Griffin-Lim (60 iterations)

Followed by the inverse pre-emphasis filter

Predicted magnitudes sharpened (power 1.3) first — models output over-smoothed spectrograms that otherwise decode to noise

Deviations from the paper

Griffin-Lim with sharpening instead of a plain inverse STFT

PCA compression for the SVR

80-dim encoding for mel spectrogram

4. Implementation Overview

- Python, built on PyTorch, scikit-learn, librosa, SciPy, cmudict
- Organised by pipeline stage:

– config.py — settings
– preprocess.py, textproc.py — text & audio -> data/dataset.npz
– models/ — svr.py, mlp.py, seq2seq_attn.py
– train.py --model svr or nn or seq2seq - saves checkpoint, loss curve, validation MSE
– postprocess.py — spectrogram -> wav
– synthesize.py — text -> wav

- Everything runs on a laptop CPU.

5. Results — Table 1

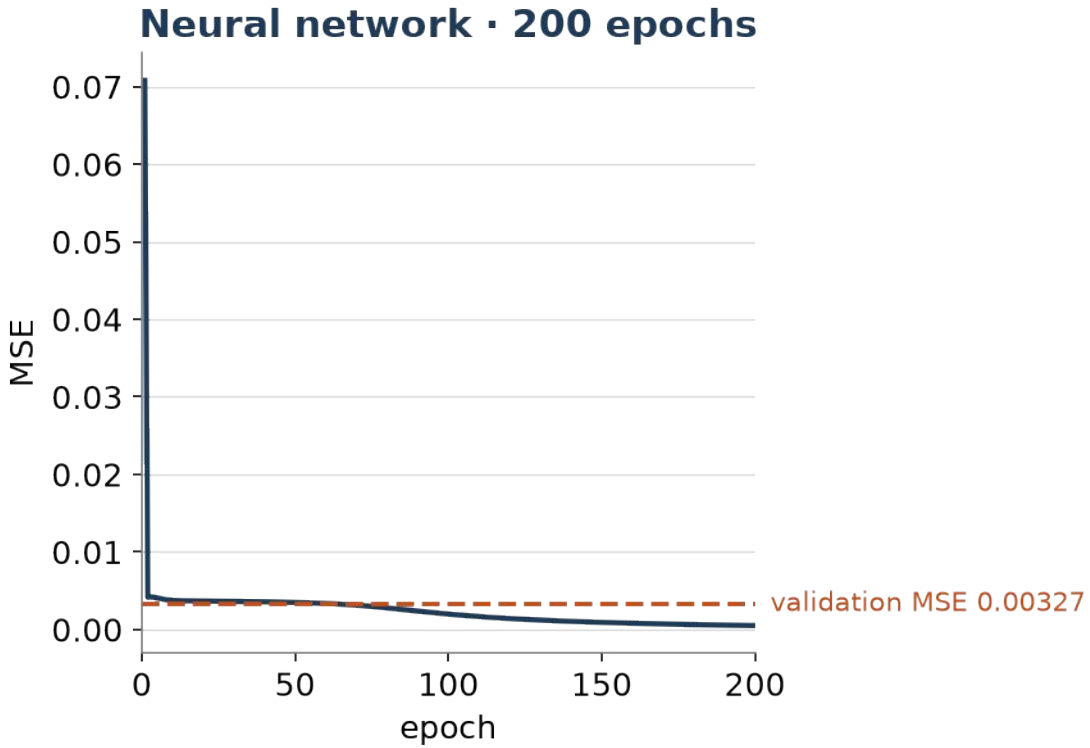
Model	Final train loss	Validation MSE	Training time
Mean-spectrogram baseline	0.00361	0.00305	–
SVR (polynomial kernel, PCA 32)	0.00218	0.00323	22 s
Neural network (200 epochs)	0.00050	0.00327	608 s
Seq2seq with attention (150 epochs)	0.00105	0.00344	4,640 s

- SVR (0.00323), NN (0.00327) and seq2seq (0.00344) are all slightly worse than the

– trivial mean-spectrogram baseline (0.00305).

- Training losses of the two neural models (0.00050, 0.00105) are 3-6.5x lower than their validation errors -> they memorise the 190 training clips.

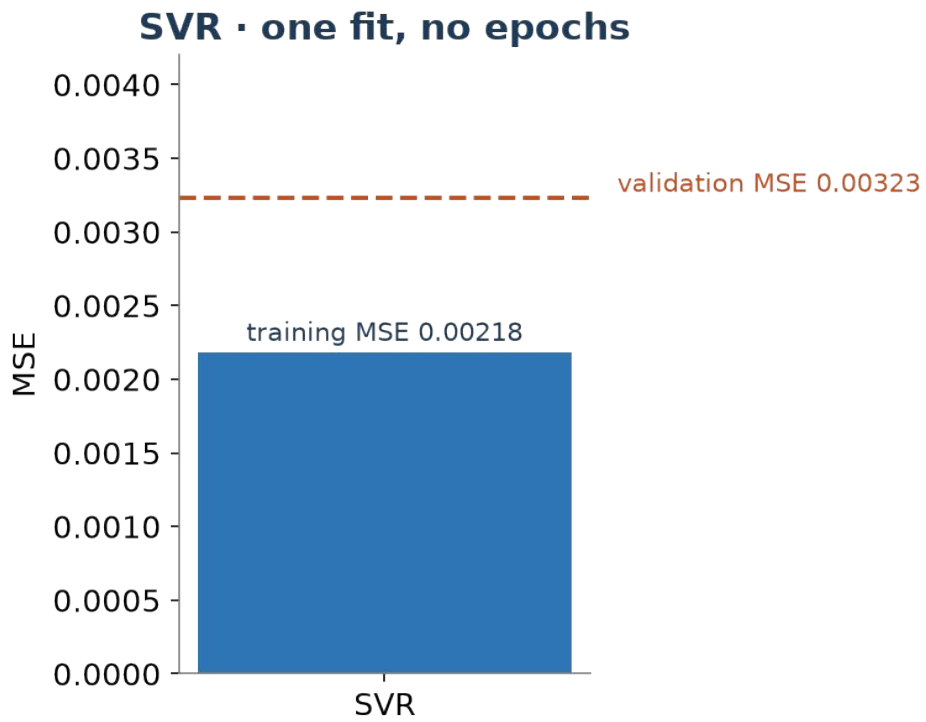
5 · Figure 2 — Training Loss vs Validation MSE

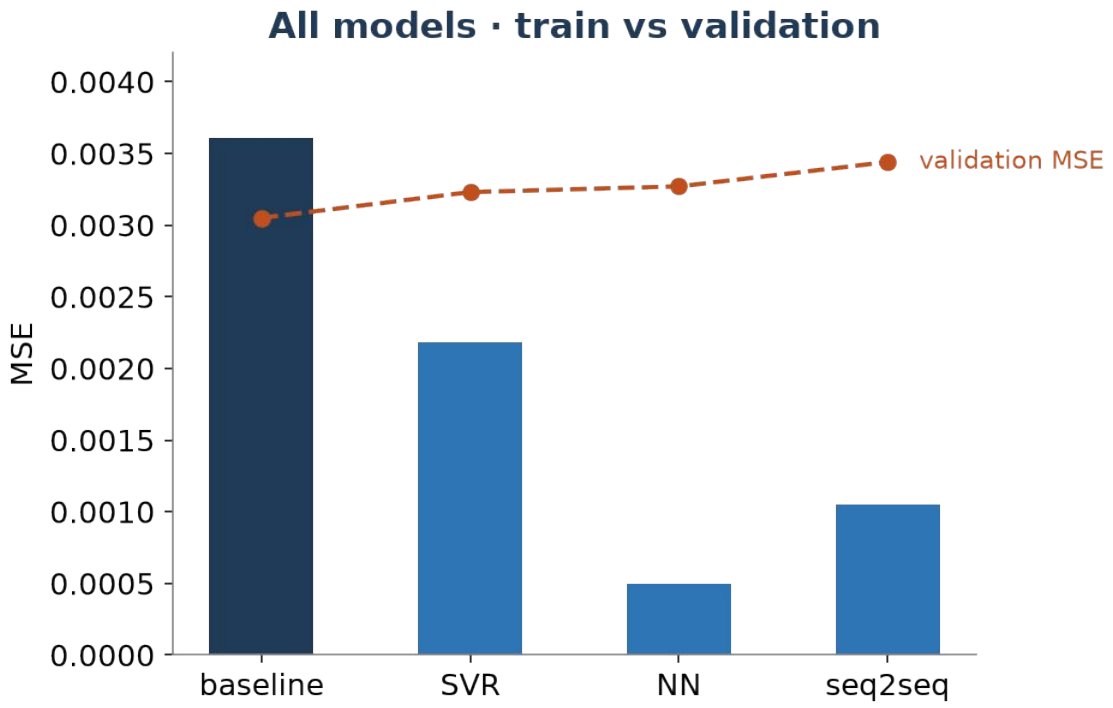




Validation error stays near 0.003 for every model while training error falls steadily. Seq2seq training loss uses teacher forcing; validation runs the decoder on its own outputs, so the two are not exactly comparable.

5 · Figure 3 — SVR Fit & Model Comparison





SVR is fitted once, so it has no loss curve — its training MSE is shown against its validation MSE (left). Training MSE for all four models (bars) against validation MSE (dashed line, right): error rises from baseline to SVR to NN to seq2seq.

5. Listening — Qualitative Findings

- Griffin-Lim on a real spectrogram gives clear audio -> reconstruction is not the bottleneck.
- Training sentence (“As a result Oswald was not hired.”, exact training symbols): all three models give clear speech.
- Unseen text (“hello world”): SVR and the neural network produce distorted speech, seq2seq produces mostly noise
- **This agrees with the paper: its models also reached low training loss but sounded unreasonable on dev and test data.**

6. Conclusion

- Built and ran the complete text -> spectrogram -> waveform pipeline, and trained all three models from the paper on the same 200-sentence LJ Speech subset.
- With 190 training sentences, none of the models generalize:

1. validation error does not beat the mean-spectrogram baseline
2. training sentences are reproduced clearly

- Differences between models on unseen text are small and dominated by overfitting the seq2seq advantage reported in the paper is not visible in our validation error.
- Main limitations: the tiny dataset, MSE as a proxy for sound quality, and the deviations in Section

6. Next Potential Steps

- Thousands of clips on a GPU
- A loss that ignores padded frames
- Consistent phoneme input
- Guided attention
- A neural vocoder in place of Griffin-Lim

References

- [1] X. Wang, Y. Yang, Y. Li, “End to End Text to Speech Synthesis,” CS 229 Project
  – Report, Stanford University, Autumn 2018.
- [2] K. Ito, L. Johnson, “The LJ Speech Dataset,” 2017.
- [3] D. Griffin, J. Lim, “Signal estimation from modified short-time Fourier
  – transform,” IEEE Trans. Acoustics, Speech, and Signal Processing, vol. 32, no. 2, 1984.
- [4] D. Bahdanau, K. Cho, Y. Bengio, “Neural machine translation by jointly learning
  – to align and translate,” ICLR 2015.